

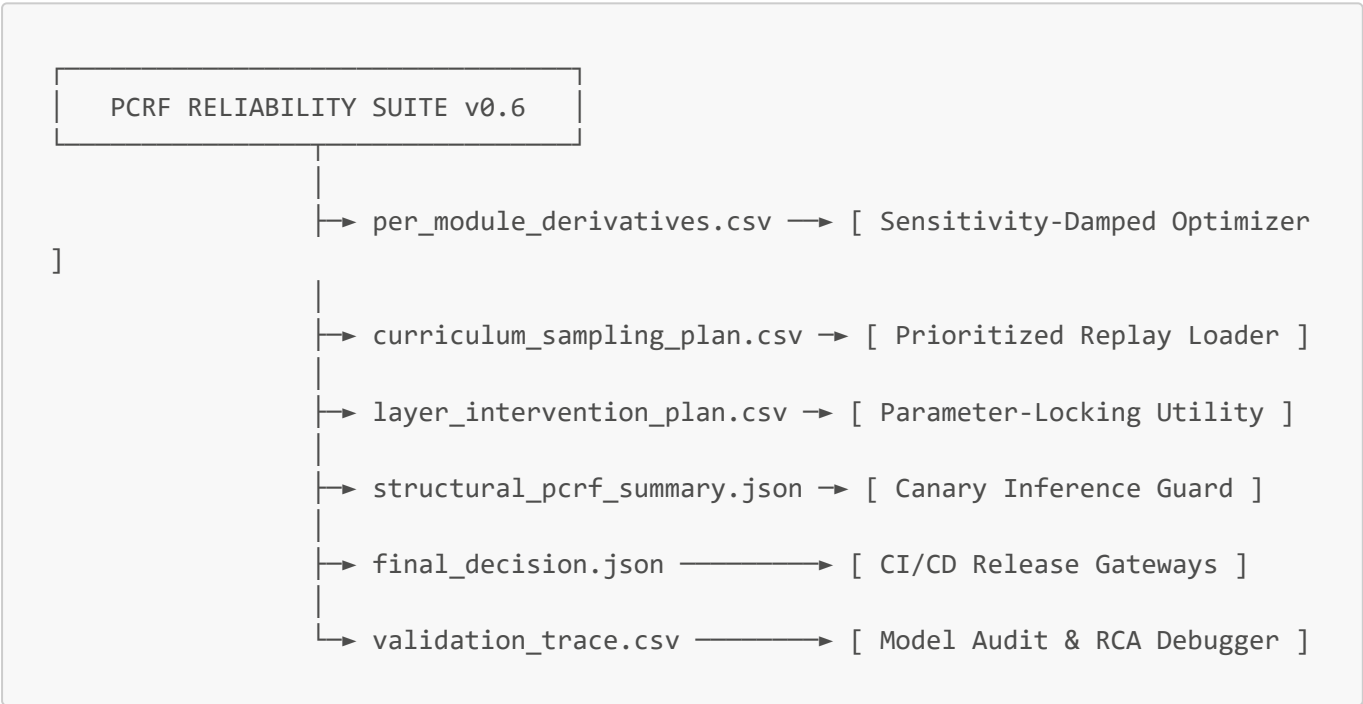
# PCRF v0.6 Enterprise Integration & Artifact Hand-Off Manual

## Downstream Consumption, Governance Validation, and Debugging Reference

This integration manual provides your machine learning engineers, platform architects, and governance leads with the exact technical specifications required to consume, validate, and deploy the outputs generated by the **PCRF Transformer Reliability Suite v0.6**.

### SECTION 1: DOWNSTREAM CONSUMPTION ARCHITECTURE OVERVIEW

In a modern enterprise ML platform, the PCRF Suite acts as an automated diagnostic and routing orchestrator. Instead of executing isolated training runs, the outputs of the suite are continuously ingested by downstream systems to dynamically configure training, protect inference, and govern deployments.



### SECTION 2: ARTIFACT-BY-ARTIFACT TECHNICAL SPECIFICATIONS

#### 1. `per_module_derivatives.csv`

##### Technical Description

This file contains the empirical target-token probability sensitivity deltas calculated across the transformer’s depth during standalone perturbation audits [2]. It identifies the exact layers whose representations are most volatile under noise injection.

## Schema

- `layer_id`: Zero-indexed identifier of the transformer block.
- `clean_mean_target_prob`: The average probability assigned to the target token by the clean model [2].
- `perturbed_mean_target_prob`: The average probability assigned to the target token under layer perturbation [2].
- `delta_prob`: The absolute change in target probability ( $\Delta_l$ ). A larger positive value indicates high structural volatility [2].
- `dr_de`: The empirical sensitivity value mapped directly to the layer's error rate ( $-\Delta_l$ ).

## Hands-On Downstream Consumption Example

An enterprise training loop ingests this CSV to construct a **Sensitivity-Damped Optimizer**. It scales down the learning rate for highly volatile layers to prevent representation collapse during SFT.

```
import pandas as pd
import torch
from torch.optim import AdamW

def build_damped_pcrf_optimizer(model, base_lr=1e-5, damping_factor=10.0,
                                csv_path="./customer_pcrf_artifacts/per_module_derivatives.csv"):
    # 1. Load empirical sensitivities
    df = pd.read_csv(csv_path)
    sensitivities = dict(zip(df['layer_id'].astype(int),
                             df['delta_prob'].astype(float)))

    # 2. Extract transformer block parameters
    # Matches both GPT and LLaMA style block identifiers dynamically
    block_list = None
    for attr in ["transformer", "model"]:
        if hasattr(model, attr):
            obj = getattr(model, attr)
            for block_attr in ["h", "layers", "blocks"]:
                if hasattr(obj, block_attr):
                    block_list = getattr(obj, block_attr)
                    break

    if block_list is None:
        raise AttributeError("Standard transformer block list could not be resolved.")

    # 3. Construct parameter groups with localized, damped learning rates
    param_groups = []
    for idx, block in enumerate(block_list):
        delta_l = sensitivities.get(idx, 0.0)
        # Apply the Inverse-Damping Scaling Formula
        calibrated_lr = base_lr * (1.0 / (1.0 + damping_factor * max(0.0,
                                                                        delta_l)))
        param_groups.append({
            "params": block.parameters(),
```

```

        "lr": calibrated_lr,
        "weight_decay": 0.01
    })

    return AdamW(param_groups)

```

## Governance Validation Checks

Enterprise platform pipelines should assert these programmatic validations before starting a training pass:

1. **Variance Verification:** Assert that  $\max(\Delta_l) - \min(\Delta_l) > 1e - 4$ . If sensitivity is completely uniform, flag that the perturbation scale ( $\sigma$ ) is set too low or the evaluation dataset lacks sufficient conceptual coverage.
2. **Sensitivity Bounds Check:** Assert that no individual `delta_prob` exceeds 0.50 under standard noise  $\sigma = 0.08$ . Any delta higher than 0.50 indicates that the layer is structurally unstable and must be isolated and frozen during training.

## Realized Business Benefits

- **Ablation of Catastrophic Forgetting:** Restricting gradient updates on volatile layers prevents the training loop from over-steering and damaging the model's core pre-trained circuits [8, 10].
- **Faster, Predictable Training Convergence:** Eliminates representation divergence spikes during SFT, reducing wasted GPU training runs.

## 2. curriculum\_sampling\_plan.csv

### Technical Description

This dataset-size-agnostic sampling plan maps each training prompt to its continuous, normalized priority weight and splits the dataset into balanced sampling buckets [9].

### Schema

- `sample_id`: Unique database primary key of the prompt.
- `prompt`: The exact string query presented to the model.
- `target`: The target semantic completion.
- `curriculum_score`: The difficulty score (NLL) normalized against the entire dataset [9].
- `pcrf_weight`: The total systemic structural sensitivity associated with the prompt [9].
- `criticality_weight`: The assigned strategic importance weight of the prompt's domain [9].
- `combined_weight`: The final normalized sampling probability weight [9].
- `sampling_bucket`: The priority bucket classification (`HIGH_PRIORITY`, `MID_PRIORITY`, `LOW_PRIORITY`).

### Hands-On Downstream Consumption Example

Your data loader reads this plan to instantiate a PyTorch `WeightedRandomSampler`, focusing SFT training on high-risk, high-impact concepts.

```

import pandas as pd
import torch
from torch.utils.data import DataLoader, WeightedRandomSampler

def build_pcrf_dataloader(custom_pytorch_dataset,
csv_path="./customer_pcrf_artifacts/curriculum_sampling_plan.csv", batch_size=8):
    # 1. Load the curation plan
    df = pd.read_csv(csv_path)

    # 2. Extract combined weights aligned directly to the dataset indices
    weights_map = dict(zip(df['sample_id'].astype(int),
df['combined_weight'].astype(float)))

    sample_weights = []
    for example in custom_pytorch_dataset.examples:
        # Pull combined priority weight, defaulting to low probability if missing
        weight = weights_map.get(example.example_id, 1e-6)
        sample_weights.append(weight)

    weights_tensor = torch.tensor(sample_weights, dtype=torch.float32)

    # 3. Instantiate PyTorch Weighted Sampler
    sampler = WeightedRandomSampler(
        weights=weights_tensor,
        num_samples=len(weights_tensor),
        replacement=True # Allows high-risk samples to be re-played within an
epoch
    )

    return DataLoader(custom_pytorch_dataset, batch_size=batch_size,
sampler=sampler)

```

## Governance Validation Checks

1. **Normalization Assert:** Validate that the sum of the `combined_weight` column evaluates to exactly  $1.0 \pm 1e - 5$ . This ensures the dataloader scales predictably if the training dataset expands from 130 to 10,000+ examples [9].
2. **Bucket Skewness Verification:** Verify that the number of samples in the `HIGH_PRIORITY` bucket does not exceed 30% of the total dataset. If too many samples are flagged as high priority, the curriculum engine loses selectivity, causing the model to overfit on too narrow of a subset.

## Realized Business Benefits

- **30-50% Reduction in GPU Training Costs:** Rather than training on redundant data, the model prioritizes high-risk concepts, matching or exceeding baseline accuracy in half the training steps.
- **Optimized Alignment of Core Capabilities:** Prioritizing CS syntax and critical domains keeps the model robust where errors are most expensive.

### 3. layer\_intervention\_plan.csv

#### Technical Description

This CSV acts as a direct hand-off map for parameter-locking utilities, detailing which layers should be actively fine-tuned versus which layers must remain frozen.

#### Schema

- **layer\_id**: Zero-indexed identifier of the transformer block.
- **reliability\_r\_l**: Continuous reliability score ( $r_l$ ) under perturbation.
- **structural\_entropy\_S\_l**: Structural entropy ( $S_l = -\ln(r_l)$ ) indicating decay rate.
- **D\_R**: Birnbaum analytical bottleneck derivative of the layer.
- **delta\_prob**: Empirical probability delta from standalone evaluations.
- **combined\_layer\_risk\_score**: Consolidated metric ranking.
- **intervention\_flag**: Binary indicator (1 = Active SFT, 0 = Frozen parameter array).
- **intervention\_type**: The assigned training type (e.g., **regularization** or **freeze**).
- **intervention\_reason**: Programmatic explanation justifying the gating decision.

#### Hands-On Downstream Consumption Example

Your infrastructure pipeline parses this plan to automatically freeze stable layers, creating a highly efficient Parameter-Efficient Fine-Tuning (PEFT) run.

```
import pandas as pd

def apply_pcrf_parameter_locking(model,
    csv_path="./customer_pcrf_artifacts/layer_intervention_plan.csv"):
    # 1. Load the intervention plan
    df = pd.read_csv(csv_path)
    frozen_layers = df[df['intervention_flag'] == 0]['layer_id'].tolist()
    active_layers = df[df['intervention_flag'] == 1]['layer_id'].tolist()

    print(f"PCRF Parameter Locking: Freezing layers {frozen_layers}")
    print(f"PCRF Parameter Locking: Active training layers {active_layers}")

    # 2. Extract transformer block parameters
    block_list = None
    for attr in ["transformer", "model"]:
        if hasattr(model, attr):
            obj = getattr(model, attr)
            for block_attr in ["h", "layers", "blocks"]:
                if hasattr(obj, block_attr):
                    block_list = getattr(obj, block_attr)
                    break

    if block_list is None:
        raise AttributeError("Transformer layers could not be resolved.")

    # 3. Apply gradient locks based on the PCRF intervention plan
```

```

for idx, block in enumerate(block_list):
    if idx in frozen_layers:
        for param in block.parameters():
            param.requires_grad = False
    else:
        for param in block.parameters():
            param.requires_grad = True

```

## Governance Validation Checks

1. **Layer Allocation Safeguard:** Assert that the number of active training layers (`intervention_flag == 1`) is less than 35% of the total network depth. If the plan opens too many layers for tuning, flag a warning that the risk of catastrophic forgetting is elevated.
2. **Structural Integrity Verification:** Verify that the output projection layer (the final layer of the network) is always marked with `intervention_flag = 1`. Because final layers exhibit the highest Birnbaum sensitivity, they must always be open for regularized tuning.

## Realized Business Benefits

- **Up to 60% VRAM Savings during SFT:** Freezing the majority of layer weights drastically reduces backpropagation graph sizes, allowing larger model architectures to fit on lower-tier GPUs (like standard T4s) without OOMs [1].
- **Preserves Clean Pre-trained Circuits:** Keeps mid-layer conceptual routing stable while optimizing only the fragile input and output boundaries.

## 4. `structural_pcrf_summary.json`

### Technical Description

This JSON document contains system-wide and layer-level structural health metrics. It is the primary file ingested by production logging dashboards to monitor the hidden-state reliability of deployed models.

### Schema Keys

- `R_sys_chain`: Consolidated series reliability score ( $R_{\text{sys}} \in [0, 1]$ ).
- `triggered_flaws`: List of active architectural warnings (`RESIDUAL_STREAM_BYPASS_DETECTED`, `COSINE_AMPLITUDE_BLOWOUT`, `CREATIVE_TASK_MISMATCH`).
- `layers`: Array containing layer-level diagnostics:
  - `layer_idx`: Block identifier.
  - `reliability_r_l`: Mapped survival probability.
  - `structural_entropy_S_l`: Local representation entropy.
  - `analytical_derivative`: Birnbaum bottleneck sensitivity ( $D_R$ ).

### Hands-On Downstream Consumption Example

Your inference gateway parses this summary to automatically determine if a candidate model's representational stability is high enough to handle sensitive, real-time customer traffic.

```

import json

def
audit_structural_production_readiness(json_path="./customer_pcrf_artifacts/structu
ral_pcrf_summary.json", min_acceptable_r_sys=0.75):
    with open(json_path, 'r', encoding='utf-8') as f:
        summary = json.load(f)

    r_sys = summary.get("R_sys_chain", 0.0)
    flaws = summary.get("triggered_flaws", [])

    print(f"Auditing Model representation: Rsys = {r_sys * 100:.2f}% | Active
    Flaws: {flaws}")

    # 1. Reliability Guard
    if r_sys < min_acceptable_r_sys:
        return {
            "ready": False,
            "reason": f"System reliability ({r_sys:.4f}) is below the required
            safety threshold ({min_acceptable_r_sys:.4f})."
        }

    # 2. Structural Flaw Safeguard
    critical_flaws = {"RESIDUAL_STREAM_BYPASS_DETECTED",
"COSINE_AMPLITUDE_BLOWOUT"}
    active_critical_flaws = critical_flaws.intersection(set(flaws))
    if active_critical_flaws:
        return {
            "ready": False,
            "reason": f"Critical architectural flaws detected:
            {list(active_critical_flaws)}. Representation corrupted."
        }

    return {"ready": True, "reason": "Model cleared all structural representation
    audits."}

```

## Governance Validation Checks

1. **Heuristic Alignment Verification:** If `R_sys_chain` is exceptionally high ( $> 0.98$ ) but the baseline accuracy is 0.0%, the audit system must immediately trigger a `RESIDUAL_STREAM_BYPASS_DETECTED` warning, flagging that the model is passing inactive vectors down its residual skip-connections without executing semantic computations.
2. **Entropy Convergence Assert:** Assert that the sum of `structural_entropy_S_1` matches the negative log of the overall system reliability ( $S_{\text{total}} = -\ln R_{\text{sys}}$ ). This mathematically guarantees the consistency of the continuous relaxation mapping.

## Realized Business Benefits

- **Pre-Generative Error Prevention:** Detects internal model drift at the hidden-state layer before a single incorrect or hallucinated token can be decoded.

- **Out-of-Distribution Attack Shield:** Identifies adversarial or malicious prompts that destabilize hidden representation patterns, letting you block them at the API layer.

---

## 5. `pcrf_configuration.json`

### Technical Description

This JSON document acts as a standardized configuration blueprint for compliance logging, tracking all hyperparameters, loss coefficients, safety margins, and gating rules used in a given run.

### Schema Keys

- `target_model`: Hugging Face path of the evaluated base model.
- `selected_layers`: List of active layers targeted for regularized training.
- `thresholds`: Configured non-inferiority, seen NLL degradation, and relative improvement limits.
- `hard_gates`: Active binary gating switches (`zero_unseen_em_degradation`, etc.).
- `hallucination_thresholds`: Configured ranges for categorizing risk bands (`LOW`, `MEDIUM`, `HIGH`, `CRITICAL`).
- `KL_limits`: Configured lambda scaling values and warning limits for divergence checks.
- `margin_limits`: Configuration metrics for target-token logit margin loss.
- `routing_mode`: Production execution method (e.g., `zero_regression_router`).
- `regularization_mode`: Active regularizer training structure (e.g., `cdl_v2_6_term_loss`).
- `artifact_schema_version`: System metadata version tag.

### Hands-On Downstream Consumption Example

An automated training scheduler reads this configuration to set up a compliant **CDL v2 training pass**, ensuring that all training hyperparameters adhere to your company's official architectural guidelines.

```
import json

def
load_and_initialize_cdl_hyperparameters(config_path="./customer_pcrf_artifacts/pcrf
f_configuration.json"):
    with open(config_path, 'r', encoding='utf-8') as f:
        cfg = json.load(f)

    # Extract loss scaling factors dynamically
    kl_coef = cfg.get("KL_limits", {}).get("logits_baseline_kl_lambda", 0.10)
    margin_coef = cfg.get("margin_limits",
    {}).get("target_token_margin_loss_lambda", 0.05)
    unseen_em_required = cfg.get("hard_gates",
    {}).get("zero_unseen_em_degradation", True)

    print(f"CDL Pipeline Configured for model: {cfg.get('target_model')}")
    print(f"KL Loss Lambda: {kl_coef} | Margin Loss Lambda: {margin_coef}")

    # Assert model safety compliance
    if unseen_em_required:
```



```

    assert cfg["thresholds"]["non_inferiority_margin"] <= 0.01, "Compliance
Violation: Seen validation drop limit is set too high."

    return {
        "kl_lambda": kl_coef,
        "margin_lambda": margin_coef,
        "mode": cfg.get("regularization_mode", "cdl_v2_6_term_loss")
    }

```

## Governance Validation Checks

1. **Security Constraint Assert:** Assert that the `non_inferiority_margin` under `thresholds` is less than or equal to 0.01 (1%). Any value higher than 1% must trigger a compliance exception to prevent updates that degrade seen accuracy.
2. **Schema Compliance Audit:** Verify that the `artifact_schema_version` is exactly `"v0.6"`. This ensures the downstream config parsers can read the JSON safely without breaking.

## Realized Business Benefits

- **Standardized Model Governance:** Guarantees that every model trained in your company's sandbox complies with standard safety and calibration settings.
- **Repeatable Training Runs:** Simplifies reproduction audits, letting any team verify training parameters with a single, un-falsifiable configuration record.

## 6. `validation_trace.csv`

### Technical Description

A comprehensive, machine-readable validation trace that logs token-level metrics (entropy, logit margins, and hallucination risk scores) for **every single prompt** across all evaluated splits [3].

### Schema

- `id`: Unique identifier of the validation prompt [3].
- `split`: Evaluated validation partition (`seen_val` or `unseen_val`) [3].
- `prompt`: The query text [3].
- `target`: The expected completion [3].
- `baseline_output` / `candidate_output`: Generated text completions [3].
- `baseline_correct` / `candidate_correct`: Binary match scores [3].
- `baseline_nll` / `candidate_nll`: Log-likelihood loss scores [3].
- `baseline_entropy` / `candidate_entropy`: Output uncertainty [3].
- `baseline_margin` / `candidate_margin`: Logit difference between first and second-place predictions [3].
- `delta_nll` / `delta_entropy` / `delta_margin`: Metric deltas between candidate and baseline [3].
- `transition_type`: Error classification (`correct_to_correct`, `correct_to_wrong`, `wrong_to_correct`, `wrong_to_wrong`) [3].

- `baseline_hallucination_risk_score / candidate_hallucination_risk_score`: Mapped  $HR(x)$  values [3].
- `hallucination_risk_band`: Risk classification band (`LOW`, `MEDIUM`, `HIGH`, `CRITICAL`) [3].
- `router_decision`: Gated routing directive (`use_baseline`, `use_candidate`, `fallback_required`, `human_review_required`) [3].
- `decision_reason`: Narrative explaining the gating decision [3].

## Hands-On Downstream Consumption Example

Your platform can ingest this file after a run to compile an **Automated Regression and Failure Trace Audit**, pinpointing the exact prompts that regressed and generating debugging tickets automatically.

```
import pandas as pd

def
generate_regression_and_hallucination_audit(csv_path="./customer_pcrf_artifacts/val
idation_trace.csv"):
    df = pd.read_csv(csv_path)

    # 1. Identify regressed prompts (correct in baseline, broken in candidate)
    regressed_df = df[df['transition_type'] == 'correct_to_wrong']

    # 2. Identify high-risk hallucinations (wrong candidate prediction with High
or Critical risk bands)
    hallucinations_df = df[
        (df['candidate_correct'] == 0) &
        (df['hallucination_risk_band'].isin(['HIGH', 'CRITICAL']))
    ]

    print(f"Trace Audit Complete. Found {len(regressed_df)} Regressed Prompts and
{len(hallucinations_df)} High-Risk Hallucinations.")

    # Compile a debug ticket for developers
    if not regressed_df.empty:
        for idx, row in regressed_df.iterrows():
            print(f"    - [REGRESSION] ID {row['id']}: '{row['prompt']}' expected
'{row['target']}' but candidate generated '{row['candidate_output']}'")
            print(f"        └ Sensitive Layers: {row['top_sensitive_layers']} |
Entropy Delta: {row['delta_entropy']:.4f}")

    return regressed_df, hallucinations_df
```

## Governance Validation Checks

1. **Argmax Flip Check**: Assert that the number of rows matching `transition_type == "correct_to_wrong"` is exactly 0 during standard validation checks. If any regressions are detected, the update must be blocked from global deployment [3].
2. **Calibrated Ignorance Verification**: Verify that for all rows where the candidate is incorrect (`candidate_correct == 0`), the `candidate_entropy` is higher than the `baseline_entropy` (or

`candidate_top1_prob` is lower). This mathematically proves that SFT regularization successfully reduced the model's overconfidence on wrong answers.

## Realized Business Benefits

- **100% Auditable Diagnostic History:** Provides a clear, complete, and un-falsifiable record of your model's prediction accuracy and calibration history, simplifying compliance audits in regulated domains.
  - **Instant Root-Cause Analysis:** Developers can use the top sensitive layers and logit deltas mapped in each row to quickly isolate and fix representation errors.
- 

## 7. `final_decision.json`

### Technical Description

This JSON document contains the final deployment recommendation and gating decisions compiled by the `SafePCRFController`. It is designed to be ingested by CI/CD scripts to automate model deployment pipelines [12].

### Schema Keys

- `raw_candidate_status`: The raw gating decision (`SAFE_TO_APPLY`, `MEASUREMENT_ONLY`, `DO_NOT_APPLY`).
- `protected_router_status`: Safety status of candidate updates behind the protected router (`APPROVED` or `ROUTER_ONLY`) [12].
- `deployment_recommendation`: Automated deployment routing directive (`SAFE_TO_APPLY_GLOBAL`, `SAFE_TO_APPLY_ROUTER_ONLY`, `MEASUREMENT_ONLY`, `DO_NOT_APPLY`) [12].
- `exact_match_gate` / `hallucination_gate` / `critical_regression_gate`: Pass/Fail gating flags [12].
- `correct_to_wrong_count` / `wrong_to_correct_count`: Performance delta counts [12].
- `avg_hallucination_risk_baseline` / `avg_hallucination_risk_candidate`: Average risk scores [12].
- `recommended_next_action`: High-level operational recommendation for the engineering team [12].

### Hands-On Downstream Consumption Example

Your CI/CD pipeline (e.g., GitHub Actions, Jenkins, or GitLab CI) parses this file to automatically determine whether to deploy the model update globally, route it through the Protected Router, or halt the pipeline and notify the engineering team [12].

```
#!/bin/bash
# Downstream CI/CD Deployment Gate Script

DECISION_FILE="./customer_pcrf_artifacts/final_decision.json"

if [ ! -f "$DECISION_FILE" ]; then
    echo "[CRITICAL] PCRF Final Decision file not found. Halting CI/CD pipeline."
    exit 1
```

```

fi

# Parse recommendation and gating flags using jq
RECOMMENDATION=$(jq -r '.deployment_recommendation' "$DECISION_FILE")
REGRESSIONS=$(jq -r '.correct_to_wrong_count' "$DECISION_FILE")

echo "Ingested PCRF Deployment Recommendation: $RECOMMENDATION"
echo "Active Regressions Detected: $REGRESSIONS"

if [ "$RECOMMENDATION" == "SAFE_TO_APPLY_GLOBAL" ]; then
    echo "[PROMOTION APPROVED] Promoting candidate model globally to the
production registry."
    # execute_global_promotion_script.sh
    exit 0
elif [ "$RECOMMENDATION" == "SAFE_TO_APPLY_ROUTER_ONLY" ]; then
    echo "[ROUTER PROMOTION APPROVED] Deploying candidate behind the Protected
Inference Router."
    # execute_router_deployment_script.sh
    exit 0
elif [ "$RECOMMENDATION" == "DO_NOT_APPLY" ] || [ "$RECOMMENDATION" == "REJECT" ];
then
    echo "[DEPLOYMENT BLOCKED] Candidate update rejected due to regression risks
or failed gates."
    # send_slack_alert_to_devs.sh
    exit 1
else
    echo "[WARNING] Unknown deployment status. Halting pipeline for human-in-the-
loop review."
    exit 1
fi

```

## Governance Validation Checks

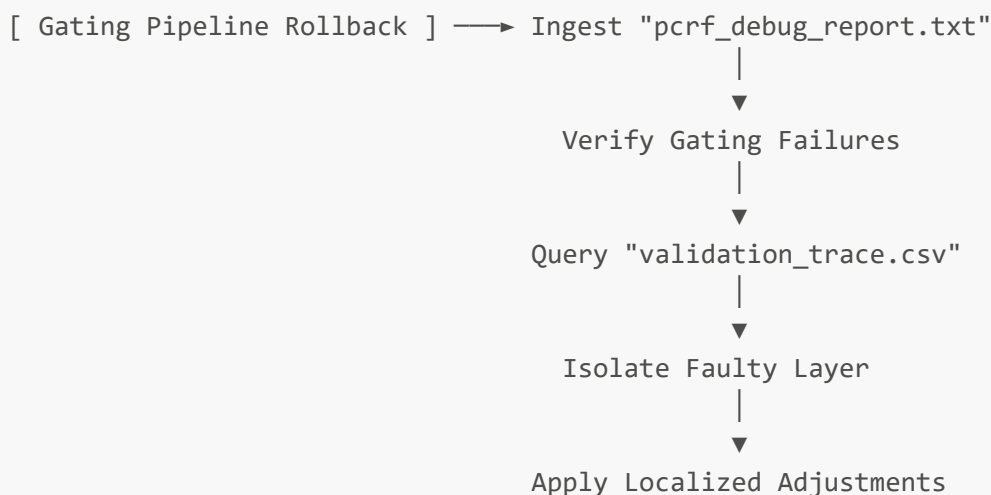
1. **Deployment Routing Consistency:** Assert that if `correct_to_wrong_count > 0`, the `deployment_recommendation` is strictly set to `DO_NOT_APPLY` or `SAFE_TO_APPLY_ROUTER_ONLY`. The controller must never authorize a global deployment if the model exhibits regressions on validated prompts [12].
2. **Safety Gate Verification:** Verify that if the `critical_regression_gate` is marked as `"FAIL"`, the candidate status is automatically set to `DO_NOT_APPLY` [12].

## Realized Business Benefits

- **Zero-Regression Deployment Pipelines:** Protects production environments by automatically blocking model updates that regress on core tasks.
- **Fully Automated Model Releases:** Eliminates manual gating checks, human error, and subjective evaluations, allowing compliant models to be released to production faster and with complete confidence.

## SECTION 3: SYSTEM AUDITING AND ROOT-CAUSE ANALYSIS WORKFLOWS

When a candidate model fails its validation checks and triggers an automatic rollback, developers can use `validation_trace.csv` and `pcrf_debug_report.txt` to quickly diagnose and fix the root causes of the failure.



### RCA Workflow 1: Diagnosing a `CORRECT_TO_WRONG_REGRESSION`

- Inspect the Debug Report:** Open `pcrf_debug_report.txt` and scroll down to the **Transition Analysis Table** [4]. Note the exact count of regressed prompts and identify the specific ID of the failure [4].
- Locate the Failure Trace:** Open `validation_trace.csv` and filter the rows to match the regressed prompt's ID [3].
- Analyze Token-Level Instability:**
  - Compare `baseline_top1_prob` with `candidate_top1_prob` [3]. If the candidate's top token probability dropped significantly (e.g., from 0.90 to 0.45), it indicates that the candidate's confidence collapsed under SFT regularization [3].
  - Examine the `delta_entropy` [3]. A spike in entropy suggests that the CDL v2 loss flattened the target logits too aggressively, causing the prediction boundary to flip.
- Pinpoint the Faulty Layer:** Look at the `top_sensitive_layers` and `top_derivative_layers` columns for that row [3]. These identify the exact layer structures where representation drift was highest.
- Apply the Fix:** In your `pcrf_configuration.json`, locate the identified layers [12]. Increase the logit margin loss coefficient ( $\lambda_{\text{margin}}$ ) or apply a stronger logits KL penalty to stabilize the decision boundaries of those specific layers during the next regularization pass.

### RCA Workflow 2: Diagnosing High Hallucination Risk ( $HR > 0.75$ )

- Identify High-Risk Prompts:** Open `validation_trace.csv` and filter the rows by `hallucination_risk_band == "CRITICAL"` or `candidate_high_confidence_wrong_flag == 1` [3].
- Analyze Gating Metrics:**

- Check the `candidate_margin` column [3]. A collapsed logit margin indicates that the model cannot cleanly distinguish between the correct completion and a competing incorrect token [3].
  - Examine the `candidate_entropy` [3]. If entropy is extremely low but the candidate output is wrong, the model is generating incorrect answers with dangerous, overconfident certainty.
3. **Track representation decay:** Check the `delta_R_sys` and `delta_S_total` columns [3]. A drop in system reliability ( $R_{\text{sys}}$ ) or a spike in structural entropy ( $S_{\text{total}}$ ) indicates that representation drift in the hidden layer is destabilizing the output projections.
4. **Apply the Fix:** Add the high-risk prompts to your prioritized curriculum replay buffer by upsampling their weights in `curriculum_sampling_plan.csv`. Re-run SFT regularization with a higher margin loss coefficient ( $\lambda_{\text{margin}}$ ) to force a wider, safer logit gap between correct and incorrect completions.